MIE353 Assignment 2, Part A

Name: _____ Student #: _____

1 Exploring the Course Management System (CMS)

Q1. How many entity classes are there in the schema (org.example.schema.CMS.*)?

There are 5 entity classes: Course, Professor, Student, Teaches and Transcript. Manifold makes one entity class for each table in the schema. The DDL file defines five tables (course, professor, student, teaches, transcript), so there are five classes. I confirmed this after compiling: the generated classes in target/classes/org/example/schema/ are CMS$Course, CMS$Professor, CMS$Student, CMS$Teaches and CMS$Transcript.

Q2. What purpose does the cms-ddl.sql file serve? Where is it used?

cms-ddl.sql is the DDL (Data Definition Language) script for the database. It contains the CREATE TABLE statements that define every table: its columns, data types, primary keys, foreign keys and constraints (e.g. NOT NULL, UNIQUE, and the 0 to 100 CHECK on grade).

It is used by Manifold-SQL. The config file src/main/resources/config/CMS.dbconfig points to it with "dbDdl": "/config/cms-ddl.sql". Because the DuckDB database is in-memory, there is no existing database file Manifold could inspect. Manifold reads this DDL instead to learn the schema at compile time (mvn compile). From it, Manifold generates the type-safe Java entity classes in org.example.schema.CMS (e.g. Student, with getters like getEmail()) and type-checks the .sql query files against the tables.

Q3. What are the foreign keys of the transcript table?

The transcript table has two foreign keys:

- studentId, which references student(studentId)
- courseId, which references course(courseId)

A transcript row can therefore only record a grade for a student and a course that already exist. Together, (studentId, courseId) also form the table's composite primary key.

Q4. How many parameters does the FindStudent query have?

One parameter, :studentId. The query is SELECT * FROM student WHERE studentId = :studentId. Manifold turns the named parameter into a method argument, which is why Main can call FindStudent.fetchOne(9002).

Q5. What format is the database stored in?

The database is a DuckDB database held in memory, not saved as a file on disk. The connection URL in CMS.dbconfig is "jdbc:duckdb:" with no file path, and "inMemory" is set to

true. The data itself comes from CSV files (students.csv, professors.csv, courses.csv, transcript.csv, teaches.csv in src/main/resources/data). These are loaded into the in-memory database with COPY ... FROM each time the program runs. Results can be written back out as CSV with COPY ... TO, as SaveStudents.sql does. Because the database is in memory, all of its data is lost when the program ends.

2 Fixing Software Problems

Problem

The DDL defines a teaches table, and the data exists in src/main/resources/data/teaches.csv. However, the starter code never loaded it. Main.loadCMSData() only loaded students, professors, courses and transcript. There was no LoadTeaches query, so the teaches table was always empty.

Fix

I followed the same pattern used for the other four tables. The fix took two changes.

Change 1: new file src/main/resources/org/example/queries/LoadTeaches.sql

This file creates the teaches table if it does not exist yet, using the same definition as in cms-ddl.sql. It then copies teaches.csv into the table:

```
CREATE TABLE IF NOT EXISTS teaches (
    profId   INT NOT NULL,
    courseId VARCHAR(20) NOT NULL,
    PRIMARY KEY (profId, courseId),
    FOREIGN KEY (profId) REFERENCES professor(profId),
    FOREIGN KEY (courseId) REFERENCES course(courseId)
);
COPY teaches FROM 'src/main/resources/data/teaches.csv';
```

Change 2: src/main/java/org/example/Main.java

I added a loadTeachesData() method, modelled on the existing load methods. It runs the LoadTeaches query inside a SQL change and commits it:

```
// Loads teaches.csv (which professor teaches which course) into the teaches table
public static void loadTeachesData() throws SQLException {
  CMS.addSqlChange(ctx -> {
    LoadTeaches.execute(ctx);
  });
  CMS.commit();
}
```

I also called this method from loadCMSData():

```
public static void loadCMSData() throws SQLException {
  loadStudentData();
  loadProfessorData();
```

```
  loadCourseData();
  loadTranscriptData();
  loadTeachesData(); // must run after professors and courses are loaded (foreign
keys)
}
```

Why the order matters

teaches has foreign keys to professor(profId) and course(courseId). Each row in teaches.csv must refer to a professor and a course that are already in the database, or the foreign key check rejects the row. loadTeachesData() is therefore called after loadProfessorData() and loadCourseData().

Testing

After adding the .sql file I ran mvn clean compile so that Manifold would generate the LoadTeaches query class. I then ran Main. To check the fix, I temporarily fetched one teaches row, Teaches.fetch(6001, "mie347h1s"), which matches the first data row of teaches.csv. It printed "6001 mie347h1s", which confirms the table is now populated. I removed this check afterwards.